LearnAble Study Notes

Depth-First Search (DFS)

Note: No external sources were supplied for this material. Therefore, these study notes are provided as general educational content rather than research-verified primary literature.

Study Notes: Depth-First Search (DFS)

1. Topic Overview

Depth-First Search (DFS) is a fundamental algorithm used for traversing or searching tree and graph data structures.

The strategy behind DFS is as simple as its name implies: the algorithm starts at a selected node (called the "root" or "source") and explores as far along each branch as possible before being forced to backtrack. Imagine exploring a maze by always choosing the path to your right until you hit a dead end, then retracing your steps to the last junction to try a different path.

2. Key Concepts and Definitions

- **Graph:** A collection of nodes (called **vertices**) connected by links (called **edges**). Graphs can be directed (one-way connections) or undirected (two-way connections).
- **Tree:** A special type of graph that has a hierarchical structure with a single root node and no cycles (loops).
- **Visited Set (or Array):** A tracking mechanism used to keep a record of which nodes the algorithm has already seen. This is crucial to prevent infinite loops in graphs that contain cycles.
- **Backtracking:** The process of stepping backward from a dead end to the most recent node that still has unexplored neighboring nodes.
- **Stack:** A Last-In, First-Out (LIFO) data structure. DFS naturally uses a stack—either the computer's **call stack** via recursion, or an **explicit stack** data structure written in code.

3. Detailed Explanations

How DFS Works (Step-by-Step)

1. **Choose a Starting Node:** Pick any arbitrary node to begin your search and mark it as "visited."
2. **Explore a Neighbor:** Look at the current node's neighbors. Pick one neighbor that has *not* been visited yet.

3. **Repeat Deeply:** Move to that neighbor, mark it as visited, and treat it as your new "current node." Repeat the process, going deeper and deeper into the graph.
4. **Hit a Dead End:** Eventually, you will reach a node that has no unvisited neighbors (a dead end).
5. **Backtrack:** Step back to the previous node and check if it has any other unvisited neighbors.

- If yes, explore that new path (repeating steps 2-3).
- If no, step back *again* to the previous node.

6. **Completion:** The algorithm terminates when all reachable nodes from the starting point have been visited.

Complexity Analysis

- **Time Complexity:** $O(V + E)$, where V represents the number of vertices (nodes) and E represents the number of edges. The algorithm visits every vertex once and traverses every edge once (in a directed graph) or twice (in an undirected graph).
- **Space Complexity:** $O(V)$. In the worst-case scenario (a graph that is just a single long line of nodes), the call stack will store all vertices. The visited set also requires memory proportional to the number of vertices.

4. Examples

Conceptual Example: A Simple Tree

Imagine a family tree rooted at **Grandpa Joe**.

- Grandpa Joe has two children: **Mary** and **John**.
- Mary has one child: **Alice**.
- John has two children: **Bob** and **Charlie**.

If we run a DFS starting from **Grandpa Joe** (and always visiting the left-most child first), the traversal order will be:

1. Start at **Grandpa Joe** (Visited: Joe)
2. Go to **Mary** (Visited: Joe, Mary)
3. Go to **Alice** (Visited: Joe, Mary, Alice) -> *Dead end! Alice has no children.*
4. Backtrack to Mary, then to **Grandpa Joe**.
5. Go to **John** (Visited: Joe, Mary, Alice, John)
6. Go to **Bob** (Visited: Joe, Mary, Alice, John, Bob) -> *Dead end!*
7. Backtrack to John, go to **Charlie** (Visited: Joe, Mary, Alice, John, Bob, Charlie) -> *Dead end!*

Final DFS Traversal Order:

```
Grandpa Joe -> Mary -> Alice -> John -> Bob -> Charlie
```

Notice how DFS completely explored Mary's entire branch down to Alice before ever looking at John's branch.

5. Summary

- **Goal:** Traverse or search graphs and trees by going as deep as possible before backtracking.
- **Data Structure:** Relies on a **Stack** (either recursive call stack or manual stack).
- **Memory Management:** Always uses a **visited list/set** to avoid infinite loops caused by cycles.
- **Best Used For:** Finding paths, testing if a graph is bipartite, cycle detection, and topological sorting.

6. Five Practice Questions with Answers

Question 1

What fundamental data structure does Depth-First Search (DFS) naturally utilize (either implicitly through recursion or explicitly)?

- **Answer:** A Stack (Last-In, First-Out / LIFO).

Question 2

Why is a "visited" array or set necessary when performing DFS on a graph?

- **Answer:** It prevents the algorithm from getting stuck in infinite loops when the graph contains cycles (loops).

Question 3

What is the worst-case time complexity of DFS in terms of vertices (V) and edges (E)?

- **Answer:** $O(V + E)$, because the algorithm visits every vertex and traverses every edge.

Question 4

True or False: DFS always finds the *shortest* possible path between a starting node and a target node in an unweighted graph.

- **Answer:** False. DFS explores paths deeply and might find a very long path first. Breadth-First Search (BFS) is the algorithm typically used to find the shortest path in unweighted graphs.

Question 5

If you run DFS on a graph, and you reach a node that has no unvisited neighbors, what action does the algorithm take next?

- **Answer:** It backtracks to the previous node in the traversal path to check for any remaining unvisited neighboring branches.